2.25　练习题

附录 A 介绍了对本章练习题有帮助的 MIPS 的模拟器。尽管模拟器可以接受伪指令，但是在要求产生 MIPS 代码的习题中，尽量不要使用伪指令。学习的目的是掌握实际的 MIPS 指令集，如果问的是指令数，则答案必须反映实际执行的指令数而非伪指令数。

有些情况必须使用伪指令（例如，当汇编时不知道真实值时，使用 la 指令）。还有些情况下，使用伪指令会更方便并使代码可读性变好（例如，li 和 move 指令）。如果因为这些原因选择使用伪指令，请在伪指令开始的地方加上一两句话，说明使用伪指令的原因。

2.1　[5]<2.2> 下面的 C 语言表达式对应的 MIPS 汇编语言代码是什么？假设变量 f、g、h 和 i 分别放在寄存器$s0、$s1 和$s2 中。要求使用最少的 MIPS 汇编指令实现。

```
f = g + (h - 5);
```

2.2　\[5\]<2.2> 写出下面的 MIPS 汇编语言程序段对应的 C 语言表达式。

```
add f, g, h
add f, i, f
```

2.3　[5]<2.2, 2.3> 写出下面的 C 语言表达式对应的 MIPS 汇编代码。假设变量 f、g、h、i 和 j 分别放在寄存器＄s0、＄s1、＄s2、＄s3 和＄s4 中。假设数组 A 和 B 的基地址分别在寄存器＄s6 和＄s7 中。

```
B[8] = A[i-j];
```

2.4　[5]<2.2, 2.3> 下面的 MIPS 汇编语言程序段对应的 C 语言表达式是什么？假设变量 f、g、h、i 和 j 分别放在寄存器＄s0、＄s1、＄s2、＄s3 和＄s4 中。假设数组 A 和 B 的基地址分别在寄存器＄s6 和＄s7 中。

```
sll  $t0, $s0, 2     # $t0 = f * 4
add  $t0, $s6, $t0   # $t0 = &A[f]
sll  $t1, $s1, 2     # $t1 = g * 4
add  $t1, $s7, $t1   # $t1 = &B[g]
lw   $s0, 0($t0)     # f = A[f]
addi $t2, $t0, 4
lw   $t0, 0($t2)
add  $t0, $t0, $s0
sw   $t0, 0($t1)
```

2.5 [5] <2.3>分别画出数据 0xabcdef12 在大端编址和小端编址的机器上是如何分布在存储器中的。(假定数据从地址 0 开始存储。）

2.6 [5]<2.4>将 0xabcdef12 转化为十进制。

2.7 [5] <2.2, 2.3> 把下面的 C 代码翻译为 MIPS 代码。假定变量 f、g、h、i 和 j 分别放在寄存器＄s0、＄s1、＄s2、＄s3 和＄s4。假定数组 A 和数组 B 的基地址分别存放在＄s6 和＄s7 中。假定数组 A 和数组 B 中的元素均为 8 字节的字：

```
B[8] = A[i] + A[j];
```

2.8 [10]<2.2, 2.3> 把下面的 MIPS 代码翻译为 C 代码。假定变量 f、g、h、i 和 j 分别放在寄存器＄s0、＄s1、＄s2、＄s3 和＄s4 中。假定数组 A 和数组 B 的基地址分别存放在＄s6 和＄s7 中。

```
addi $t0, $s6, 4
add  $t1, $s6, $0
sw   $t1, 0($t0)
lw   $t0, 0($t0)
add  $s0, $t1, $t0
```

2.9 [20]<2.3,2.5>对于练习 2.8 中的每条 MIPS 指令，写出操作码（OP）、源操作数（RS）、目标操作数（RD）及功能字段（funct field）的值（value）。对于Ⅰ型指令，写出立即数字段的值。对于 R 型指令，写出目的寄存器（RD）字段的值。

2.10　假定寄存器＄s0 和＄s1 分别存放数值 0x80000000 和 0xD0000000。

2.10.1　[5]<2.4> 执行下面汇编代码后，＄t0 的值是多少？

```
add $t0, $s0, $s1
```

2.10.2　[5]<2.4> ＄t0 中的值是期望的结果还是发生溢出后的结果？

2.10.3　[5]<2.4> 对于上面定义的寄存器＄s0 和＄s1 的内容，执行下面汇编代码后，＄t0 的值是多少？

```
sub $t0, $s0, $s1
```

2.10.4　[5]<2.4> ＄t0 中的值是期望的结果还是发生溢出后的结果？

2.10.5　[5]<2.4> 对于上面定义的寄存器＄s0 和＄s1 的内容，执行下面汇编代码后，＄t0 的值是多少？

```
add $t0, $s0, $s1
add $t0, $t0, $s0
```

2.10.6　[5]<2.4> ＄t0 中的值是期望的结果还是发生溢出后的结果？

2.11 假定＄s0 中的值为 128_{10}。

2.11.1　[5]<2.4> 对于指令 add ＄t0, ＄s0, ＄s1，求使结果产生溢出的＄s1 的值的范围。

2.11.2　[5]<2.4> 对于指令 sub ＄t0, ＄s0, ＄s1，求使结果产生溢出的＄s1 的值的范围。

2.11.3　[5]<2.4> 对于指令 sub ＄t0, ＄s1, ＄s0，求使结果产生溢出的＄s1 的值的范围。

2.12 [5]<2.4，2.5> 写出下面的二进制数值对应的类型和汇编语言指令：
$0000\ 0010\ 0001\ 0000\ 1000\ 0000\ 0010\ 0000_2$。提示：从图 2-20 可能会获取帮助。

2.13 [5]<2.4,2.5> 给出下面指令的类型和十六进制表示:

sw　＄t1, 32(＄t2)

2.14 [5]<2.5>写出用下面 MIPS 字段描述的指令的类型、汇编语言指令和二进制表示：

```
op=0, rs=3, rt=2, rd=3, shamt=0, funct=34
```

2.15 [5]<2.5>写出用下面 MIPS 字段描述的指令的类型、汇编语言指令和二进制表示：

```
op=0x23, rs=1, rt=2, const=0x4
```

2.16 假设可以将 MIPS 寄存器文件扩展到 128 个寄存器，并将指令集中的指令数扩展为原来的 4 倍。

2.16.1　[5]<2.5> 这将如何影响 R 型指令的每个位字段的大小？

2.16.2　[5]<2.5> 这将如何影响 I 型指令的每个位字段的大小？

2.16.3　[5]<2.5,2.10> 在提出的这两种变化中，每种变化如何减少一个 MIPS 汇编程序的

大小？另一方面，如何增大一个 MIPS 汇编程序的大小？

2.17　假设如下寄存器内容：

```
$t0=0xAAAAAAAA, $t1=0x12345678
```

2.17.1　[5]<2.6> 对于以上的寄存器内容，执行下面的指令序列后＄t2 的值是多少？

```
sll $t2, $t0, 44
or $t2, $t2, $t1
```

2.17.2　[5]<2.6> 对于以上的寄存器内容，执行下面的指令序列后＄t2 的值是多少？

```
sll $t2, $t0, 4
andi $t2, $t2, -1
```

2.17.3　[5]<2.6> 对于以上的寄存器内容，执行下面的指令序列后＄t2 的值是多少？

```
srl $t2, $t0, 3
andi $t2, $t2, 0xFFEF
```

2.18　[5]<2.6> 找出完成如下功能的最短的 MIPS 指令序列：从寄存器＄t0 中提取第 16 位到第 11 位，然后使用这些位替换寄存器＄t1 的第 31 位到第 26 位，保持其他位不变。（确认在进行测试时使用$t0=0 和$t1=0xffffffff。这样做可能会揭示一个被经常忽略的问题）

2.19 [5]<2.6> 写出可用来实现下面伪指令的 MIPS 指令集的最小子集：

```
not $t1, $t2        // bit-wise invert
```

2.20 [5]<2.6> 对于下面的 C 语言表达式，写一个能够完成同样操作的最短 MIPS 汇编指令程序段。假设＄t1=A, ＄t2=B,＄s1 是 C 的基地址。

```
A = C[0] << 4;
```

2.21 [5]<2.7> 假设＄t0 中存放数值 0x00101000，在执行下列指令后＄t2 的值是多少？

```
        slt $t2, $0, $t0
        bne $t2, $0, ELSE
        j DONE
ELSE: addi $t2, $t2, 2
DONE:
```

2.22 假设程序计数器（PC）被设置为 0x2000 0000。

2.22.1 [5]<2.10>如果使用 MIPS 的 jump-and-link（jal）指令，可以转移到的地址范围是多少？（也就是说，跳转指令执行之后 PC 值可能的集合是什么？）

2.22.2 [5]<2.10>如果使用 MIPS 的相等则分支（beq）指令， 可以转移到的地址范围是多少？（也就是说，分支指令执行之后 PC 值可能的集合是什么？）

是否可以使用 MIPS 的跳转（j）指令将 PC 设置为地址 0x4000 0000？是否可以使用 MIPS 的相等则分支（beq）指令将 PC 设置为该地址？

2.23 假如提出了一条称为 rpt 的新指令，该指令将循环条件检查和计数器递减合并到一条指令中。例如，rpt $s0,loop 的功能如下：

```
if (x29 > 0) {
          x29 = x29 - 1;
          goto loop
}
```

2.23.1 [5]<2.7,2.10>如果要在 MIPS 指令集中实现该条指令，哪种指令格式最适合？

2.23.2 [5]<2.7>如果使用 MIPS 集中的指令实现该操作，请写出长度最短的指令序列。

2.24 考虑如下的 MIPS 循环：

```
LOOP: slt  $t2, $0,  $t1
      beq  $t2, $0,  DONE
      subi $t1, $t1, 1
      addi $s2, $s2, 2
      j    LOOP
DONE:
```

2.24.1 [5]<2.7> 假设寄存器 $t1 的初始值为 10，假设 $s0 初始值为 0，则循环执行完毕时寄存器 $s0 的值是多少？

2.24.2 [5]<2.7>对于上面的循环体，写出等价的 C 代码。假定寄存器 $s1、 $s2、 $t1 和 $t2 分别为整数 A、B、i 和 temp。

2.24.3 [5]<2.7>假定寄存器 $t1 的初始值为 N，上面的 MIPS 汇编循环执行了多少条指令？

2.25 [5]<2.7>将下面的 C 代码翻译为 MIPS 汇编代码。要求使用的指令数目最少。假设值 a、b、i 和 j 分别存放在寄存器 $s0、 $s1、 $t0 和 $t1 中。另外假设寄存器 $s2 中存放着数组 D 的基地址。

```
for(i=0; i<a; i++)
   for(j=0; j<b; j++)
      D[4*j] = i + j;
```

2.26 [5]<2.7> 实现习题 2.25 中的 C 代码用了多少条 MIPS 汇编指令？如果变量 a 和 b 分别初始化为 10 和 1，并且 D 中所有元素初始化为 0，将整个循环执行完成时，一共执行了多少条 MIPS 指令？

2.27 [5]<2.7>将下面的循环翻译成 C 代码。假定寄存器 $t1 中存放 C 语言级的整数 i， $s2 中存放 C 语言级的整数 result， $s0 存放整数数组 MemArray 的基地址。

```
       addi $t1, $0, 0
LOOP: lw   $s1, 0($s0)
       add  $s2, $s2, $s1
       addi $s0, $s0, 4
       addi $t1, $t1, 1
       slti $t2, $t1, 100
       bne  $t2, $s0, LOOP
```

2.28 [10]<2.7>将习题 2.27 中的循环重写以减少执行的 MIPS 指令。提示：注意变量 i 只用于循环控制。

2.29 [30]<2.8>使用 MIPS 汇编实现下面的 C 代码。提示：栈指针必须保持为 16 的整数倍。

```
int fib(int n){
    if (n==0)
       return 0;
    else if (n == 1)
       return 1;
    else
       return fib(n-1) + fib(n-2);
}
```

2.30 [20]<2.8>对于每一次函数调用，画出调用后栈的内容。假定栈指针被初始化为 0x7ffffffc，寄存器的使用情况和图 2-11 相同。

2.31 [20]<2.8>将下面的函数 f 翻译成 MIPS 汇编语言。如果需要使用寄存器 $t0 到 $t7，请从编号小的寄存器开始使用。假设函数 func 的声明为“int f(int a，int b)；”，函数 f 的代码如下：

```
int f(int a, int b, int c, int d){
  return func(func(a,b),c + d);
}
```

2.32[5]<2.8>请问这个函数可以使用尾调用优化吗？如果不能，请说明原因。如果能，请说明优化前后执行 f 的指令数的差别。

2.33 [5] <2.8>在习题 2.31 中函数 f 返回之前，可以知道寄存器＄t5、＄s3、＄ra 和＄sp 的内容吗？（注意，我们知道函数 f 的全部，但是我们只知道函数 func 的声明。)

2.34 [30] <2.9>用 MIPS 汇编语言写一段代码将包含十进制正整数和负整数的 ASCII 码的数串转换成整数。在程序中使用寄存器＄a0 保存由数字 0～9 组成的非空串的地址。程序应该计算与这个数字串等值的整数，并将这个整数存放在寄存器＄v0 中。如果在字符串的任意位置出现非数字字符，程序停止并将-1 存入＄v0。例如，如果寄存器＄a0 指向 3 字节的序列 $50_{10},52_{10},0_{10}$(非终结的字符串“24”)，当程序停止的时候，寄存器＄v0 中的值应该是 24_{10}。

2.35 [5]<2.9>对于如下代码：

```
lbu $t0, 0($t1)
sw $t0, 0($t2)
```

假设寄存器＄t1 中存放地址 0x1000 0000，该地址存放的数据为 0x11223344。

2.35.1 [5]<2.3,2.9>在大端地址的机器中，存储在 0x10000004 处的数据是多少？

2.35.2 [5]<2.3,2.9>在小端地址的机器中，存储在 0x10000004 处的数据是多少？

2.36 [5]<2.10>请编写能产生 32 位常数 0010 0000 0000 0001 0100 1001 0010 0100_2（此处本版原文应该有误，译者参考本书第五版给出常数值）的 MIPS 代码，并将值存储到寄存器＄t1 中。

2.37 [10]<2.11>写出能够实现下面 C 代码的 MIPS 汇编语言代码，要求使用 MIPS 的 ll/sc 指令。该 C 代码完成了取最大值（set max）的操作，变量 shvar 中包含一个共享变量的地址，如果 x 大于 shvar 指向的值，则使用 x 替换该共享变量：

```
void setmax(int* shvar, int x) {
  // Begin critical section
  if (x > *shvar)
     *shvar = x;
  // End critical section}
}
```

2.38 [5]<2.11>以练习题 2.37 中的代码为例，解释当两个处理器同时执行这段临界区域时，将发生什么情况？假设每个处理器执行一条指令正好需要一个周期。

2.39 假设给定处理器的算术指令的 CPI 是 1，取数/存数指令的 CPI 是 10，分支指令的 CPI 是 3。假设一个程序由 5 亿条算术指令、3 亿条取数/存数指令和 1 亿条分支指令组成。

2.39.1 [5]<1.6，2.13>假设向指令集中添加了新的、功能更强的算术指令。通过使用这些功能更强大的算术指令平均可以减少程序执行所需要的 25%的算术指令，而时钟周期的开销增长了 10%。请问这是好的设计选择吗？为什么？

2.39.2 [5]<1.6，2.13>假设可以找到一种可以使算术指令性能达到原来两倍的方法。请问机器的整体加速是多少？假设可以找到一种可以使算术指令性能达到原来 10 倍的方法，那么机器的性能整体加速又是多少？

2.40 假设一给定程序共执行了 70%的算术指令、10%的取数/存数指令和 20%的分支指令。

2.40.1 [5]<2.21>假设执行一条算术指令、取数/存数指令和分支指令分别需要 2 个周期、6 个周期和 3 个周期，求平均 CPI。

2.40.2 [5]<1.6，2.13> 在取数/存数指令和分支指令执行时间不变的情况下，如果要使性能提升 25%，则算术运算指令的平均执行时间应该为多少？

2.40.3 [5]<1.6，2.13> 在取数/存数指令和分支指令执行时间不变的情况下，如果要

使性能提升 50%，则算术运算指令的平均执行时间应该为多少？

2.41 [10]<2.21>假设 MIPS ISA 中包含了比例偏移（scaled offset）的寻址方式，该寻址方式与 2.19 节（图 2-35）中给出的 x86 相应的寻址方式相同。如果要使用“比例变址取数指令”（scaled offset loads）进一步减少实现练习 2.4 中功能的汇编指令数量，请给出你的思路。

2.42 [10]<2.21>假设 MIPS ISA 中包含了比例偏移（scaled offset）的寻址方式，该寻址方式与 2.19 节（图 2-35）中给出的 x86 相应的寻址方式相同。如果要使用“比例变址取数指令”（scaled offset loads）进一步减少实现练习 2.7 C 代码的汇编指令数量，请给出你的思路。

小测验答案

2.2 MIPS，C，Java

2.3 2）.非常慢

2.4 第一问：2）-8_{10} 第二问：4）$18\ 446\ 744\ 073\ 709\ 551\ 608_{10}$

2.5 第一问：4）sub ＄t2,＄t0,＄t1 第二问：28_{16}